

第一题:

1. DFS (G)

for each vertex do
 $color[u] \leftarrow white$

Init Stack (S)

for each vertex do

 if $color[u] = white$

$color[u] = Gray$

 push (S, u)

 while !empty(S) do

 pop (S, v)

$color[v] \leftarrow Dark$

 for each $u \in V$ do

 if $color[k] = white$ then

$color[k] \leftarrow Gray$

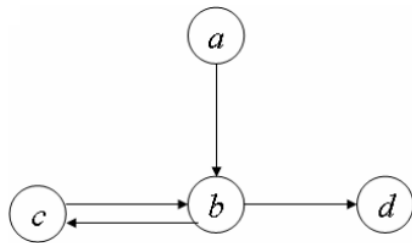
 push (S, k)

第二题:

```
2. prim ( MGraph G, VertexType u, p)
{
    k = LocateVex (G, u)
    for j=0 in G.vexnum do           // 辅助数组初始化
        if (j != k)
            closedge[j] = {u, G.arcs[k][j].adj}.
    closedge[k].lowcost = 0           // 初始化 u = {u}
    for i=1 in G.vexnum do
        { k = minimum (closedge)
          print (closedge[k].adjvex, G.vexs[k]) // 输出生成树的边
          closedge[k].lowcost = 0           // k 并入 U 集
          for j=0 in G.vexnum do
              if (G.arcs[k][j].adj < closedge[j].lowcost)
                  closedge[j] = {G.vexs[k], G.arcs[k][j].adj}
          }
        closedge[p].lowcost = 0
        k = maximum {closedge [p~q]}       // p, q 为不连通的两顶点
        delete (closedge[k].adjvex, G.vexs[k])
    }
```

思想: 把修改的边加入到最小生成树中, 在一条含有修改边的回路, 去掉权值最大, 即可。

第三题:



以 a 为源点。则 $d(a)=1, d(b)=2, d(c)=3, d(d)=4, d(c)>d(d)$, 且从 c 到 d 存在路径, 但是 d 不是 c 的后继。

第四题:

Prim 算法的时间复杂度为 $O(|E|\lg|V|)$, Kruskal 算法的时间复杂度为 $O(|E|\lg|E|)$, 要使 Prim 算法慢于 Kruskal 算法, 即要求 $|E|\lg|V| > |E|\lg|E|$, 可得 $V > E$ 。又因为这两个算法是用来求图的最小生成树, 所以 $E \geq V-1$ 。综上所述, 当 $E=V-1$ 时, Prim 算法要慢于 Kruskal 算法。故存在这样的图。